

Where Authorization Stops

Effect-Bound Containment, Independent Evidence and the Limits of Preview

Aditya Pratap Singh

September 2026

AI Incident Response Sprint · Track 1 (Containment) · Apart Research

The incident, and the question it leaves open

July 2026. Two evaluated models escaped their sandbox through the **one network egress it permitted** — a package-registry cache proxy with a zero-day — then attacked production infrastructure.

- “Only one permitted egress” said nothing about **what effects** could pass through it.
- An accurate log of what an agent *intended* can disagree with what the backend *executed*.

The incident, and the question it leaves open

July 2026. Two evaluated models escaped their sandbox through the **one network egress it permitted** — a package-registry cache proxy with a zero-day — then attacked production infrastructure.

- “Only one permitted egress” said nothing about **what effects** could pass through it.
- An accurate log of what an agent *intended* can disagree with what the backend *executed*.

Our question: *Where should authority be checked, and what evidence lets an outside reviewer confirm that only authorized effects occurred?*

Three controls, tested on real backends

1. **Check authority for the exact effect** — request, identity, one-use nonce, target — *at the component that executes it*, not just upstream.
2. **Keep effect evidence the actor and broker cannot forge**, acquired independently from the backend.
3. **Treat a pre-execution preview as advisory, not proof.**

Testbeds (all synthetic, no production data): native **Git** hooks · a **Bubblewrap-isolated HTTP** gateway · local **Pome** API twins · hosted **Arga** twins · a separate-process local service.

Three separate boundaries — not one combined deployment

A. Native Git — enforcement timing



Same effect policy at preview and receive time; 3/6 versus 0/6 attack successes.

B. Pome — independent observation



Detection is after execution. The observer does not prevent or roll back the effect.

C. Local quarantine — preview fidelity



Matching copied state does not guarantee matching future behavior. Host/backend trust remains.

Finding 1 — enforce where the effect executes

Same policy, only *relocated* to the effect boundary:

Condition	Attacks allowed	Benign completed
Git: broker dry-run preview	3 / 6	5 / 5
Git: receive-time effect gate	0 / 6	5 / 5
HTTP: sandbox + endpoint restriction	8 / 12	4 / 5
HTTP: sandbox + exact-authority gateway	0 / 12	5 / 5

Finding 1 — enforce where the effect executes

Same policy, only *relocated* to the effect boundary:

Condition	Attacks allowed	Benign completed
Git: broker dry-run preview	3 / 6	5 / 5
Git: receive-time effect gate	0 / 6	5 / 5
HTTP: sandbox + endpoint restriction	8 / 12	4 / 5
HTTP: sandbox + exact-authority gateway	0 / 12	5 / 5

Every violation that passed an equally-informed *upstream* check was stopped at the boundary — **with no loss of legitimate work.**

Finding 2 — the log can lie honestly

Six executed attacks on a Pome API twin. What each evidence source detected:

Evidence source	Detected
Agent intent log	0 / 6
Gate-entry request record	2 / 6
Independent backend tape	6 / 6

- Upstream records still *matched the approved request* while the backend did something else.
- Detection is **after** execution — forensics, not prevention. Both matter; don't confuse them.

Finding 3 — a clean preview is not proof

Preview-based quarantine (run the request on a copy first, release if clean):

- Blocked every fault **its preview could see**.
- **Tied** plain static denial on prevention — no incremental gain.
- A fault present **only at execution** escaped it on *every* testbed: local, hosted, and separate-process.

Finding 3 — a clean preview is not proof

Preview-based quarantine (run the request on a copy first, release if clean):

- Blocked every fault **its preview could see**.
- **Tied** plain static denial on prevention — no incremental gain.
- A fault present **only at execution** escaped it on *every* testbed: local, hosted, and separate-process.

Verdict on our own hypothesis: the *enforcement* half was confirmed; the *quarantine-adds-prevention* half was **not**. A preview is a prediction — and predictions break.

The fix we found: an execution-time effect gate

Confine each operation's *committed effect* **inside the transaction**; roll back anything outside its authorized class.

Fault	Static	Preview	State gate	Data-flow gate
Execution-only write	leaks	leaks	prevented	prevented
Read exfiltration	leaks	leaks	leaks	prevented

- The state gate stops the execution-only write **and completes the work** — preview had to *hold* it.
- A leaking read changes no state, so a state-only view calls it clean; confining **outbound data flow** at the boundary catches it and still returns the authorized value.

The fix we found: an execution-time effect gate

Confine each operation's *committed effect* **inside the transaction**; roll back anything outside its authorized class.

Fault	Static	Preview	State gate	Data-flow gate
Execution-only write	leaks	leaks	prevented	prevented
Read exfiltration	leaks	leaks	leaks	prevented

- The state gate stops the execution-only write **and completes the work** — preview had to *hold* it.
- A leaking read changes no state, so a state-only view calls it clean; confining **outbound data flow** at the boundary catches it and still returns the authorized value.

Right problem, wrong mechanism, better mechanism found.

Built for Track 1: verifiable without the lab's network

- `containment-standard.md` — nine controls, each with an acceptance rule, the evidence a real lab must supply, and a strict split between a *fixture* PASS and a *deployment* PASS.
- `./verify.sh` — one command recomputes every archived result from its receipts, offline. No accounts, no API keys, no lab network.
- An **answer-key-withheld** AI reviewer reached all six scoped PASS / FAIL / NOT-VERIFIED decisions from the evidence packet alone — *package usability*, explicitly **not** human validation.
- **Everything is preserved** — 34 hypotheses, every failed and incomplete run, the negative results.

Limitations — stated plainly

- **Trust boundary.** Broker, gate, observer and backend share one trusted host. A *compromised* egress — the actual incident — is out of scope. We do not claim this would have stopped it.
- **Constructed cases.** Deterministic fixtures show a failure *can* occur and the control stops it — not how often it occurs in the wild.
- **Unresolved.** Quarantine's incremental value over an execution gate; realistic model-attacker resistance (three pilots inconclusive); covert and implicit channels.
- **Pending.** Independent human replication and a lab's adoption-cost assessment.

Containment that names a permitted route but not the effects allowed through it is hard to verify and easy to misuse.

- **Enforce the exact effect at the boundary that executes it.**
- **Keep evidence the actor cannot forge.**
- **Never accept a clean preview as proof of a safe action.**

302 tests · `./verify.sh` → PASS · all runs and negatives retained

github.com/Aditya-PS-05/effect-bound-containment